```
void increment(std::atomic<int>* var_to_inc,read_values* values)
{
    while(!go)                                          <-- Spin, waiting for the signal (3)
        std::this_thread::yield();
    for(unsigned i=0;i<loop_count;++i)
    {
        values[i].x=x.load(std::memory_order_relaxed);
        values[i].y=y.load(std::memory_order_relaxed);
        values[i].z=z.load(std::memory_order_relaxed);
        var_to_inc->store(i+1,std::memory_order_relaxed);    <-- (4)
        std::this_thread::yield();
    }
}

void read_vals(read_values* values)
{                                                       <-- (5) Spin, waiting for the signal
    while(!go)
        std::this_thread::yield();
    for(unsigned i=0;i<loop_count;++i)
    {
        values[i].x=x.load(std::memory_order_relaxed);
        values[i].y=y.load(std::memory_order_relaxed);
        values[i].z=z.load(std::memory_order_relaxed);
        std::this_thread::yield();
    }
}
```

This is a really simple program in essence. You have three shared global atomic variables ❶ and five threads. Each thread loops 10 times, reading the values of the three atomic variables using `memory_order_relaxed` and storing them in an array. Three of the threads each update one of the atomic variables each time through the loop ❹, while the other two threads just read. Once all the threads have been joined, you print the values from the arrays stored by each thread ❼.

The atomic variable `go` ❷ is used to ensure that the threads all start the loop as near to the same time as possible. Launching a thread is an expensive operation, and without the explicit delay, the first thread may be finished before the last one has started. Each thread waits for `go` to become `true` before entering the main loop ❸, ❺, and `go` is set to `true` only once all the threads have started ❻.

One possible output from this program is as follows:

```
(0,0,0),(1,0,0),(2,0,0),(3,0,0),(4,0,0),(5,7,0),(6,7,8),(7,9,8),(8,9,8),
     (9,9,10)
(0,0,0),(0,1,0),(0,2,0),(1,3,5),(8,4,5),(8,5,5),(8,6,6),(8,7,9),(10,8,9),
     (10,9,10)
(0,0,0),(0,0,1),(0,0,2),(0,0,3),(0,0,4),(0,0,5),(0,0,6),(0,0,7),(0,0,8),
     (0,0,9)
(1,3,0),(2,3,0),(2,4,1),(3,6,4),(3,9,5),(5,10,6),(5,10,8),(5,10,10),
     (9,10,10),(10,10,10)
(0,0,0),(0,0,0),(0,0,0),(6,3,7),(6,5,7),(7,7,7),(7,8,7),(8,8,7),(8,8,9),
     (8,8,9)
```